

BAŞLIK: PYTHON İLE KENDİ API'NİZİ İNŞA ETMEK: FASTAPI İLE PRODUCTION-GRADE REST API

TÜR: Uygulamalı/Teknik Detaylı Anlatım

BÖLÜM 1: NEDEN FASTAPI?

Python ekosisteminde bir Web API inşa etmek için en bilinen 3 seçenek Flask, Django REST Framework ve FastAPI'dir. Bu modülde FastAPI'yi seçmemizin somut, teknik gerekçeleri vardır:

- Otomatik Veri Doğrulama (Validation):** FastAPI, Pydantic kütüphanesini kullanarak, bir isteğin gövdesinin/parametrelerinin doğru TİPTE ve doğru KURALLARDA olup olmadığını OTOMATİK kontrol eder — elle `if not isinstance(...)` kontrolleri yazmaya gerek kalmaz.
- Otomatik Dokümantasyon:** Yazdığınız kod ÜZERİNDEN, hiçbir ek çaba harcamadan, interaktif Swagger UI ve ReDoc dokümantasyonu otomatik üretilir (bkz. 22.7).
- Native Async Desteği:** Python'ın `async/await` yeteneklerini tam destekler; bu da I/O yoğun (çok sayıda eşzamanlı ağ/veritabanı isteği bekleyen) API'lerde ciddi performans avantajı sağlar.
- Tip İpuçları (Type Hints) Üzerinden Çalışması:** Modern Python'ın tip ipuçları (`def f(x: int) -> str:`) doğrudan çerçevenin (framework) çekirdeğinde kullanılır — bu hem kod okunabilirliğini artırır hem editörlerin (VS Code/PyCharm) otomatik tamamlama kalitesini yükseltir.

BÖLÜM 2: TEMEL BİLEŞENLER

2.1. Path Operations (Rota Tanımları)

FastAPI'de bir uç nokta (endpoint), bir fonksiyonun üzerine bir "dekoratör" (decorator) eklenerek tanımlanır: `@app.get("/urunler/{urun_id}")`. Süslü parantez içindeki `{urun_id}` bir PATH PARAMETRESİDİR — URL'nin bir parçası olarak gelir ve fonksiyon parametresi olarak otomatik yakalanır. Sorgu parametreleri (query params) ise fonksiyon imzasında path'te belirtilmeyen ekstra parametreler olarak tanımlanır ve FastAPI bunları otomatik olarak `?anahtar=deger` formatından çıkarır.

2.2. Pydantic Modelleri: Şema Tanımlama ve Doğrulama

Pydantic, bir `BaseModel` sınıfından türeyen Python sınıfları üzerinden veri şemaları tanımlamanızı sağlar. Bir istek gövdesi bu şemaya UYMUYORSA (örn. zorunlu bir alan eksikse, bir sayı yerine metin gönderilmişse), FastAPI OTOMATİK olarak 422 `Unprocessable Entity` hatası döner — bu kontrolü YAZMANIZA bile gerek kalmaz. Pydantic ayrıca `Field(gt=0)` (sıfırdan büyük olmalı), `EmailStr` (geçerli e-posta formatı) gibi zengin doğrulama kuralları da sunar.

2.3. Bağımlılık Enjeksiyonu (Dependency Injection)

FastAPI'nin en güçlü — ve başta biraz soyut gelen — özelliği `Depends()` sistemidir. Bir uç noktanın ihtiyaç duyduğu ortak işlevleri (örn. "mevcut kullanıcıyı token'dan çöz", "veritabanı oturumu aç") ayrı, yeniden kullanılabilir fonksiyonlara taşımanızı sağlar. Örneğin kimlik doğrulama gerektiren HER uç noktaya `current_user: User = Depends(get_current_user)` parametresini eklemeniz yeterlidir; FastAPI bu fonksiyonu otomatik çalıştırıp sonucu enjekte eder. Bu, kodun TEKRARINI (DRY - Don't Repeat Yourself)

önler ve test edilebilirliği artırır (testlerde bu bağımlılık kolayca "sahte" (mock) bir versiyonla değiştirilebilir).

2.4. Hata Yönetimi

Bir iş kuralı ihlal edildiğinde (örn. "bu ID'li kitap bulunamadı"), `HTTPException(status_code=404, detail="Kitap bulunamadı")` fırlatılır; FastAPI bunu otomatik olarak doğru durum koduyla, standart bir JSON hata formatına çevirir. Daha karmaşık uygulamalarda, özel exception sınıfları ve `@app.exception_handler()` ile TÜM API genelinde tutarlı bir hata formatı (bkz. Bölüm 4) uygulanabilir.

BÖLÜM 3: 22.6.2 PROJESİNİN MİMARİSİ — KÜTÜPHANE YÖNETİM API'Sİ

22.6.2 dosyasında, gerçek bir production API'nin sahip olması gereken TÜM katmanları içeren bir "Kütüphane Yönetim Sistemi" API'si inşa edeceğiz:

1. Veri Katmanı (SQLAlchemy ORM + SQLite): Kitaplar ve kullanıcılar, gerçek bir veritabanı tablosunda saklanır (in-memory Python listesi DEĞİL) — SQLAlchemy ORM ile tanımlanan modeller, 15-SQL modülünde öğrendiğimiz ilişkisel veritabanı kavramlarını doğrudan Python nesnelere bağlar.
2. Şema Katmanı (Pydantic): Veritabanı modelleri (SQLAlchemy) ile API'nin dışarıya gösterdiği şemalar (Pydantic) BİLİNÇLİ OLARAK AYRILIR — bu ayrım, veritabanı yapısındaki bir değişikliğin API sözleşmesini kırmasını önler ve hassas alanların (örn. şifre hash'i) yanlışlıkla dışarı sızmasını engeller.
3. Kimlik Doğrulama Katmanı (JWT): Kullanıcılar /token uç noktasından giriş yapar, bir JWT alır (bkz. 22.4.1, Bölüm 2.3) ve sonraki isteklerde bu token'ı Authorization: Bearer başlığıyla gönderir.
4. İş Mantığı Katmanı: CRUD (Create, Read, Update, Delete) işlemleri, yetkilendirme kontrolleri ("bu kitabı sadece ekleyen kullanıcı silebilir").
5. Sunum Katmanı (Path Operations): Yukarıdaki katmanları birleştiren, HTTP isteklerini karşılayan uç noktalar; sayfalama (bkz. 22.3, Bölüm 4) ve filtreleme destekli bir liste uç noktası dahil.

3.1. Neden SQLite ve TestClient?

Bu projeyi, bu repository'nin diğer tüm notebook'ları gibi TAMAMEN OFFLINE ve TEKRAR ÜRETİLEBİLİR (reproducible) tutmak için: (1) harici bir veritabanı sunucusu (PostgreSQL/MySQL) yerine, tek bir dosyada çalışan SQLite kullanılır — kurulum gerektirmez; (2) API'yi gerçekten bir ağ portunda çalıştırıp requests ile dışarıdan çağırmak yerine, FastAPI'nin kendi test aracı olan TestClient'ı kullanılır — bu, isteği GERÇEK bir HTTP çağrısı gibi (aynı doğrulama, aynı yönlendirme mantığı) işler ama süreç içi (in-process) çalıştığı için hem çok daha hızlıdır hem de ağ/port çakışması gibi ortam sorunlarından tamamen bağımsızdır. Production'da bu API, uvicorn main:app komutuyla gerçek bir sunucu olarak çalıştırılır — bunu da notebook içinde ayrıca göstereceğiz.

BÖLÜM 4: TUTARLI HATA FORMATI VE CORS AYARLARI

İyi bir API, HER hata durumunda AYNI YAPIDA bir JSON döndürmelidir — istemci kodunun her hata türü için farklı bir ayrıştırma (parsing) mantığı yazmasını önler. Örneğin: {"hata": {"kod": "KİTAP_BULUNAMADI", "mesaj": "...", "detaylar": null}}. Bu yaklaşım, RFC 7807 ("Problem Details for HTTP APIs") standardından ilham alır ve büyük API'lerin (Stripe, GitHub) hata formatlarında yaygın olarak görülür.

CORS (bkz. 22.4.1, Bölüm 4.2) ayarı, FastAPI'de CORSMiddleware ile yapılır — projemizde, sadece belirli originlerin (örn. bir frontend geliştirme sunucusu) API'ye tarayıcı üzerinden erişebilmesine izin vereceğiz; production'da ASLA allow_origins=["*"] (herkese izin ver) kullanılmamalıdır — bu, CSRF benzeri saldırılara kapı aralar.

SONUÇ

FastAPI'nin temel bileşenlerini (path operations, Pydantic doğrulama, dependency injection, hata yönetimi) ve production kalitesinde bir API'nin katmanlı mimarisini öğrendik. Sıradaki dosyada (22.6.2), bu teoriyi uçtan uca çalışan, test edilmiş bir Kütüphane Yönetim API'sine dönüştüreceğiz.